

INSTITUTO FEDERAL DE SANTA CATARINA

CÂMPUS GASPAR

CURSO ANÁLISE E DESENVOLVIMENTO EM SISTEMAS

DISCENTES:

GABRIEL DZIECINNY FERREIRA

VINÍCIUS DA SILVA LANSER.

1 PADRÕES CRIACIONAIS	3
1.1 Singleton (DB.java)	3
1.2 Builder (Flight.java)	3
2 PADRÃO ESTRUTURAL	3
2.1 Facade (FlightFacade.java)	3
3 PADRÃO COMPORTAMENTAL	4
3.1 Observer (EntityObserver.java)	4
Referências	5

1 PADRÕES CRIACIONAIS

1.1 Singleton (DB.java)

Utilizado quando o sistema necessita de uma única instância global de acesso a um recurso centralizado, controlando estritamente seu ponto de acesso.

No projeto, a classe DB atua simulando a persistência em memória. Múltiplas instâncias gerariam coleções de dados separadas e inconsistências. O Singleton garante que todo o fluxo manipule exatamente o mesmo mapa de armazenamento (storage) via método estático.

```
private static DB instance;
public static DB getInstance() {
    if (instance == null) { instance = new DB(); }
    return instance;
}
```

1.2 Builder (Flight.java)

Indicado para a criação de objetos complexos compostos por múltiplos atributos, permitindo uma construção passo a passo e independente da representação final.

A entidade Flight demanda diversas propriedades (número, companhia, origem, destino, capacidade). O uso da classe interna estática Builder evita a criação de construtores gigantescos com ordenação confusa, tornando a inicialização fluida e legível por encadeamento.

```
Flight flight = new Flight.Builder()
    .withNumber("G3 1500").withCompany("GOL")
    .withFrom("GRU").withTo("NVT").build();
```

2 PADRÃO ESTRUTURAL

2.1 Facade (FlightFacade.java)

Fornece uma interface unificada e simplificada para um subsistema complexo ou um conjunto de classes inter-relacionadas.

Persistir uma entidade exige validações de negócio, checagem de concorrência de IDs e execução do ciclo de vida (doSave). A AbstractEntityFacade e sua filha FlightFacade encapsulam todos esses passos, expondo à classe principal (Program) um método limpo de inserção.

```
facade.insert(flight); // Oculta validações e chamadas ao DB
```

3 PADRÃO COMPORTAMENTAL

3.1 Observer (EntityObserver.java)

Define uma dependência um-para-muitos entre objetos, de modo que quando um muda de estado, seus dependentes são notificados automaticamente.

Aplicado para permitir que o sistema reaja dinamicamente às inserções de entidades sem acoplamento rígido. A Facade mantém uma lista de EntityObserver e dispara os métodos de gatilho antes e após a persistência física dos dados.

```
for (EntityObserver obs : observers) {  
    obs.onBeforeInsert(entity);  
}
```

Referências

SHVETS, Alexander. Padrões de Projeto. Refactoring.Guru, [s.l.], 2014-2026.
Disponível em: <https://refactoring.guru/pt-br/design-patterns>. Acesso em: 28 jun.
2026.